

Arquitetura Santa Cecília

Sistemas escaláveis, seguros e preparados para o mundo real

8 de outubro de 2026 · Palestra de 40 min · Demo de 10 min (plano de 5 min)

Repositório público

<https://github.com/DevKaue/arquitetura-santa-cecilia>

Referência: main. Importação por URL dentro da demonstração curta.

Como usar este guia

1. Resumo executivo: alinhar as decisões e os slides com a chefia.
2. Roteiro: acompanhar ações, falas, parâmetros e transições.
3. Cartão do apresentador: consultar a sequência rapidamente.
4. Perguntas e respostas: conduzir a discussão por tema.

Visão de apresentação: v10 · 13 blocos · 13 relações. Mapa completo na v9; 3 laboratórios mantidos.

Resumo executivo para revisão

Evento: palestra de arquitetura de software, Santa Cecília, 8 de outubro de 2026. **Palestra completa:** 40 minutos. **Demonstração:** 10 minutos no Build Arch Desktop, com alternativa de 5 minutos. **Repositório:** [DevKaue/arquitetura-santa-cecilia](#). **Projeto principal:** Arquitetura Santa Cecília, visão de apresentação v10; mapa completo preservado na v9.

1. Objetivo da demonstração

Transformar os conceitos dos slides em decisões visíveis: distribuir leituras, proteger a compra, retirar trabalho posterior do tempo de resposta e identificar o gargalo antes de aumentar a infraestrutura. A plateia acompanha três jornadas: consulta ao catálogo, compra transacional e processamento após a compra.

O exemplo mantém o caso de catálogo e compras apresentado no material da palestra. O nome do projeto é **Arquitetura Santa Cecília**. A visão de apresentação reúne 13 blocos e 13 relações em três jornadas, com as réplicas de leitura agrupadas. O mapa completo de 19 blocos/25 relações permanece na v9. Os três laboratórios foram mantidos; o roteiro compara apenas Labs 01 e 03, se houver tempo.

2. Problema e premissas

Premissa	Valor do exercício	Uso na explicação
Tráfego normal	2.000 requisições/min	Referência para comparação
Pico de campanha	20.000 requisições/min	Aumento de 10 vezes
Perfil do negócio	95% leitura; 5% escrita	Separar catálogo e compra
Cache HIT ilustrativo	90% das leituras	Calcular alívio no banco, sem confundir com roteamento do simulador

Com essas premissas, o pico produz 19.000 leituras e 1.000 escritas por minuto. Um HIT de 90% deixa 1.900 leituras chegando ao banco. Essa conta explica uma hipótese de arquitetura; a simulação de carga não aplica automaticamente a proporção 95/5 nem o cache HIT.

3. Decisões centrais

Tema	Decisão	Consequência a discutir
Leitura	CDN, cache-aside e réplicas	Ganho de latência/capacidade com atraso tolerado no catálogo
Compra	Autorizar o dono; reservar no writer	Evitar venda sem estoque e acesso indevido a pedidos
Pagamento	Idempotência e reconciliação	Timeout mantém resultado desconhecido até confirmação
Assíncrono	Outbox, relay, consumidor e DLQ	Entrega pode repetir; o efeito precisa ser controlado
Operação	Métricas, canary e rollback	Detectar degradação e reduzir o impacto de mudança
Recuperação	Writer único e região secundária	RPO/RTO propostos exigem teste de restauração e failover

4. Evidência disponível para a demonstração

No cenário temporal de pico 4x, com a mesma entrada, os Labs 01 e 02 produzem latência estimada de 895 ms e erro estimado de 68%. A API cresce de uma para seis instâncias, mas o banco permanece limitante. No Lab 03, a capacidade cadastrada do banco aumenta e os valores passam a 48 ms e 0,05%.

Como exercício complementar, fora do roteiro principal, na carga ao vivo 10x, são geradas 10.000 requisições em 30 segundos virtuais. Após o limite de drenagem, Labs 01/02 deixam aproximadamente 2.650 pendentes; o Lab 03 termina sem pendências. São resultados do modelo do Build Arch, sem representar benchmark HTTP ou SLA real.

O Compose declara nove serviços. A inferência estática pelo motor do Build Arch identificou 23 blocos e 22 relações, incluindo redes, volume, imagens e abstrações de software. Esse resultado permite explicar a revisão humana de um mapa extraído do GitHub.

5. Alinhamento com os slides

Slides	Ponto de revisão
3-4	Retomar o caso e explicitar que os números são premissas
11	Consistência depende das operações e do isolamento adotado
14	TLS pode terminar na entrada; nova sessão protege o backend
18	Reserva de estoque ocorre no checkout; notificação pode esperar
19-21	Relacionar segurança, observabilidade, canary e limite de escala
22-23	Diferenciar simulação temporal 4x, carga ao vivo 10x e gargalo compartilhado

6. Escopo e revisão proposta

O repositório publica o modelo, os manifests e as notas técnicas. Os servidores são um esqueleto para inferência: catálogo sintético, health endpoints e checkout com resposta 501. Pagamento, consumidor e relay reais não estão implementados. A sintaxe do Compose foi validada; os containers não foram executados.

A distribuição sugerida é 26 minutos de slides/conclusão, 10 de demonstração e 4 de perguntas. A demo não exige montagem de componentes, navegação por versões nem modelagem de dados ao vivo.

Para revisar, abrir o backup principal, acompanhar o roteiro curto e conferir: separação leitura/compra, reserva concorrente, idempotência, evento de confirmação de compra, limites de retry, DLQ, redundância e metas de recuperação. Revisar especialmente as correções propostas para os slides 14 e 18 antes do ensaio conjunto.

Roteiro enxuto da demonstração

Palestra completa: 40 minutos. Demonstração no Build Arch: 10 minutos. Se o tempo apertar, usar a versão de 5 minutos ao fim deste roteiro. O foco é conectar as decisões dos slides ao desenho; a configuração da ferramenta fica pronta no ensaio.

Distribuição sugerida dos 40 minutos

Tempo da palestra	Conteúdo
0-5 min	Problema, contexto e premissas
5-15 min	Escalabilidade, consistência e resiliência
15-23 min	Evolução da arquitetura, segurança e operação
23-33 min	Demonstração de 10 min no Build Arch
33-36 min	Decisões finais e conclusão
36-40 min	Perguntas

Preparação antes de entrar em cena

Abrir **Arquitetura Santa Cecília**, em **Trabalho atual · v10**, e enquadrar o mapa. A visão compacta tem três jornadas. O desenho completo de 19 blocos/25 relações está preservado na **v9**; não é necessário percorrer o histórico durante a apresentação.

Ensaiar a importação de <https://github.com/DevKaue/arquitetura-santa-cecilia>, referência **main**. Deixar os Labs 01 e 03 conferidos com os parâmetros do cartão. Conferir **Salvo agora**. Não montar caixas, schema ou conexões ao vivo.

Sequência principal: 10 minutos

Tempo da demo	Mostrar	Mensagem
0:00-1:00	Visão geral	Leitura e compra têm necessidades diferentes
1:00-4:00	Três jornadas	Cache para leitura; writer para reserva; fila para pós-compra
4:00-6:00	Importar URL do GitHub	Código oferece evidências para revisão
6:00-9:00	Labs 01 e 03	Investir no gargalo muda o resultado
9:00-10:00	Voltar ao mapa e concluir	Medir e validar as decisões no sistema real

0:00-1:00 - Retomar o caso

Ação: mostrar a visão compacta, já aberta em 2D.

Fala: “Este desenho reúne o caso dos slides. A consulta ao catálogo pode aceitar algum atraso. A compra precisa reservar estoque e evitar repetição de cobrança. O que vem depois da compra pode sair do tempo de resposta.”

Retomar 95% leitura/5% escrita e pico 10x em uma frase. As premissas já foram explicadas nos slides.

1:00-4:00 - Explicar três jornadas

Leitura: apontar **Catálogo > Cache / Réplicas de leitura**. HIT responde pelo cache; MISS consulta uma réplica. As duas réplicas estão agrupadas para a apresentação.

Compra: apontar **Checkout > Writer + standby / Pagamento externo**. Autorização do dono, idempotência e reserva concorrente no writer. A integração de pagamento ocorre fora da transação SQL; timeout exige reconciliação.

Pós-compra: apontar **Publicador da outbox > Fila > Notificações / DLQ**. Pedido e intenção de evento são gravados juntos. A notificação de compra confirmada depende de pagamento reconciliado; a entrega pode repetir, por isso o consumidor controla duplicatas.

Fala: “Cada caminho tem uma exigência. Distribuímos leitura, protegemos a decisão de compra e deixamos tarefas posteriores em uma fila.”

Mencionar em uma frase que identidade, observabilidade, CI/CD e recuperação também fazem parte da arquitetura e estão detalhadas na v9 e nas notas. Não abrir a modelagem de dados nesta sequência.

4:00-6:00 - Trazer a arquitetura do GitHub

Ação: **Analisar projeto > GitHub público**. Colar <https://github.com/DevKaue/arquitetura-santa-cecilia>, referência **main**, executar e mostrar o rascunho/evidências. Não editar o mapa inferido durante a palestra.

Fala: “O repositório já declara serviços, redes e dependências. A ferramenta extrai evidências, mas precisamos conferir se a relação é inicialização, comunicação ou uma hipótese.”

Aponte apenas um exemplo: o relay depende de PostgreSQL e RabbitMQ. O resultado verificado contém 23 blocos/22 relações para nove serviços do Compose, incluindo imagens, redes e volume. Não significa 23 microsserviços.

Se a análise demorar mais de 45 segundos, continuar a explicação no mapa principal, sem ficar aguardando em silêncio. Se a Internet falhar, mostrar a evidência publicada e esclarecer que a importação ao vivo não terminou.

6:00-9:00 - Uma comparação de gargalo

Ação: mostrar o resultado do **Lab 01** e do **Lab 03**, com o mesmo cenário ensaiado. Parâmetros: **2.000 req/min, Pico, 4x, 10 min virtuais, Nenhuma falha**. Executar somente se os controles já estiverem prontos; se faltar tempo, usar a tabela do cartão.

Resultado do modelo	Lab 01	Lab 03
Capacidade cadastrada do banco, req/min	3.000	20.000
Latência estimada, ms	895	48
Erro estimado	68%	0,05%

Fala: “Mais APIs continuam chegando ao mesmo banco. O Lab 02, que fica como apoio, demonstra isso. A melhoria precisa atingir o limitador, com otimização ou capacidade medida.”

Declarar uma vez: “Esses números são saídas do modelo, não medições de produção.” Não repetir o Lab 02 nem fazer carga ao vivo, falha injetada ou passeio em 3D nesta sequência.

9:00-10:00 - Fechar e retornar aos slides

Ação: retornar à **Arquitetura Santa Cecília**.

Fala: “O desenho organiza as responsabilidades; o repositório ajuda a encontrar evidências; as métricas ajudam a testar hipóteses. O próximo passo real é validar concorrência, segurança, capacidade e recuperação.”

Mencionar que o backup e o repositório guardam as decisões. Retornar à conclusão da palestra. Exportação e navegação de histórico ficam para depois.

Plano de 5 minutos se o tempo apertar

Tempo	Ação
0-1 min	Apresentar a visão geral
1-3 min	Explicar leitura, compra e pós-compra
3-4:30 min	Importar do GitHub e apontar uma evidência
4:30-5 min	Concluir e retornar aos slides

Sem simulação ao vivo. A comparação dos Labs fica em uma resposta à plateia ou no material de apoio. Se a rede não colaborar, manter o mapa já aberto e seguir o fechamento.

Material mantido para ensaio e perguntas

Os três Labs, o schema de oito tabelas, os relatórios de carga/inferência, as notas técnicas e o mapa completo da v9 continuam nos mesmos arquivos. A carga ao vivo 10x usa 2.000 usuários × 10 ações/min por 30 s; Labs 01/02 deixam aproximadamente 2.650 pendentes após drenagem, e Lab 03 termina sem pendências. Esse exercício fica fora da demo principal.

O diagnóstico e os motores usam heurísticas. A visão compacta agrupa componentes e omite relações de controle/operação para facilitar a leitura; não representa a implantação completa nem um fluxo exato de simulação.

Cartão do apresentador

Palestra: 40 min · Demo: 10 min · Plano curto: 5 min · Build Arch Desktop: v10

Tempo da demo	Ação	Frase-chave
0-1 min	Mapa já aberto	"Leitura e compra têm exigências diferentes."
1-4 min	Três jornadas	"Cache para leitura, writer para reserva e fila para pós-compra."
4-6 min	Importar URL do GitHub	"O código oferece evidências; o mapa precisa de revisão."
6-9 min	Comparar Labs 01/03	"Mais APIs continuam chegando ao mesmo banco."
9-10 min	Voltar ao mapa	"Medir, validar e registrar a decisão."

URL pronta para copiar

<https://github.com/DevKaue/arquitetura-santa-cecilia> Referência **main**. Analisar projeto > **GitHub público**.

Apontar relay > PostgreSQL/RabbitMQ; não editar o mapa inferido. Se demorar mais de 45 s, continuar a explicação no mapa principal.

Comparação pronta

Temporal: **2.000 req/min** · Pico · **4x** · **10 min virtuais** · **Nenhuma falha**.

Resultado	Lab 01	Lab 03
Latência estimada	895 ms	48 ms
Erro estimado	68%	0,05%

O Lab 02 mantém o resultado do Lab 01 mesmo com seis APIs. Não precisa executar os três. Uma frase sobre o limite: "Os números são saídas do modelo, não medições de produção."

Se só houver 5 minutos

Mapa 1 min > três jornadas 2 min > GitHub 1,5 min > fechar 30 s. Omitir simulação. Histórico, schema, carga ao vivo, 3D e exportação ficam fora do roteiro principal.

Antes de começar

Conferir **Trabalho atual** · **v10**, **Salvo agora**, enquadramento e importação da URL. Deixar Labs 01/03 ensaiados. Mapa completo na **v9**; laboratórios e notas mantidos para perguntas. Sugestão de orçamento: 26 min de slides/conclusão, 10 min de demo e 4 min de perguntas.

Perguntas e respostas para discussão

Reservar cerca de **4 minutos** da palestra de 40 minutos para perguntas. Usar a **resposta breve**; manter o aprofundamento como apoio para conversa posterior. Priorizar as perguntas 2 (gargalo), 5 (estoque) e 14 (inferência), conforme a plateia. A demonstração principal tem 10 minutos e não precisa cobrir este caderno inteiro.

A. Escolha e evolução da arquitetura

1. É necessário começar com microserviços?

Resposta breve: não. Um monólito modular pode atender a fase inicial com menos custo operacional.

Aprofundamento: separar implantação faz sentido quando volume, autonomia da equipe, frequência de mudança ou isolamento de falhas justificam observabilidade, comunicação distribuída e operação adicionais. O mapa ensina responsabilidades; não impõe um estilo único.

2. Como decidir o que escalar?

Resposta breve: medir a dependência limitante antes de aumentar instâncias.

Aprofundamento: olhar saturação, filas, latência medida, erros, queries, índices, pool de conexões e concorrência. No Lab 02, seis APIs continuam chegando ao mesmo banco. O Lab 03 altera a capacidade cadastrada no limitador; a intervenção real precisa de medição.

3. Qual o custo dessa arquitetura?

Resposta breve: o painel apresenta valores didáticos; um orçamento real depende do uso e do provedor.

Aprofundamento: região, tráfego de saída, requests, processamento, armazenamento, observabilidade, suporte e operação precisam entrar na conta. Autoscaling exige teto e proteção das dependências compartilhadas.

B. Dados, cache e compra

4. O que significa o perfil 95/5?

Resposta breve: é a hipótese de negócio: 95% das chamadas leem e 5% escrevem.

Aprofundamento: no pico de 20.000 req/min, seriam 19.000 leituras e 1.000 escritas. Um HIT de 90% é uma segunda hipótese e deixa 1.900 leituras chegando ao banco. O motor de ramificações não aplica automaticamente esses percentuais.

5. Como impedir que dois clientes comprem a última unidade?

Resposta breve: decidir a reserva no writer com atualização condicional dentro da transação.

Aprofundamento: atualizar somente se houver estoque suficiente; fazer rollback de todos os itens se um falhar. Usar ordem estável para itens e tratar deadlock/retry com idempotência. Um teste concorrente deve provar que o saldo não fica negativo.

6. Cache ou réplica podem decidir a reserva?

Resposta breve: a tela pode aceitar atraso; a reserva usa o dado autoritativo.

Aprofundamento: TTL e replicação podem exibir estado antigo. A API de compra precisa conferir disponibilidade, preço e autorização no domínio apropriado. Falha do cache exige fallback limitado para não provocar avalanche no banco.

7. Standby aumenta a capacidade de escrita?

Resposta breve: não. O modelo mantém um writer e um standby para alta disponibilidade.

Aprofundamento: réplicas A/B atendem leituras do catálogo com atraso possível. HA, distribuição de leitura e capacidade de escrita são decisões distintas. A promoção deve impedir que o writer antigo continue aceitando escrita.

C. Resiliência e processamento assíncrono

8. Retry sempre melhora a disponibilidade?

Resposta breve: retry sem limite pode aumentar a sobrecarga e repetir efeitos.

Aprofundamento: timeout, backoff com jitter, orçamento finito e idempotência são necessários. Em pagamento, timeout significa resultado desconhecido: consultar estado ou reconciliar antes de compensar a reserva.

9. Outbox garante exatamente uma entrega?

Resposta breve: não. Ela grava pedido e intenção de publicar na mesma transação local.

Aprofundamento: o relay pode publicar, receber confirmação e cair antes de marcar o evento. A repetição exige deduplicação por evento/consumidor. O evento de compra confirmada nasce após reconciliação do pagamento; criar pedido não autoriza enviar confirmação de compra.

10. Como evitar e-mail duplicado?

Resposta breve: o efeito externo também precisa de uma política de idempotência.

Aprofundamento: gravar `processed_events` não torna atômica a chamada ao serviço de e-mail. Exigir idempotência do provedor ou documentar a tolerância a duplicata e a reconciliação. ACK deve acompanhar a política de efeito durável.

11. Para que serve a DLQ?

Resposta breve: retirar mensagens que esgotaram tentativas e permitir investigação e reprocessamento controlado.

Aprofundamento: monitorar idade/volume, definir responsável e retenção, corrigir a causa e repetir com deduplicação. DLQ sem processo operacional vira acúmulo de problema.

D. Segurança, operação e recuperação

12. WAF e token válido bastam para segurança?

Resposta breve: não. A aplicação também precisa validar entrada e autorizar cada recurso.

Aprofundamento: conferir o dono do pedido, menor privilégio no banco e nos serviços, rede privada para dados/broker, segredos por ambiente e logs sem tokens/cartão. WAF atua na borda; não conhece todas as regras do domínio.

13. Qual a disponibilidade real? E a recuperação de região?

Resposta breve: disponibilidade exige medição; RPO/RTO exigem teste de desastre.

Aprofundamento: definir SLI, período de observação e SLO. As metas propostas de $RPO \leq 5$ min e $RTO \leq 30$ min precisam de ensaio de restore, promoção, fencing e roteamento. CDN mundial não torna o writer active-active nem substitui backup.

E. Build Arch e repositório

14. Por que a arquitetura inferida é diferente do desenho final?

Resposta breve: os manifests revelam recursos e dependências; o desenho final incorpora decisões de negócio e operação.

Aprofundamento: a inferência inclui redes, volume, imagens e abstrações com possíveis sobreposições. Revisar evidências e separar inicialização de chamada real. Capacidade, segurança, consistência e fluxo de pagamento não são comprovados por `depends_on`.

15. A nota do diagnóstico e os números da simulação garantem o sistema?

Resposta breve: são indicadores do modelo, úteis para discutir hipóteses.

Aprofundamento: o diagnóstico examina blocos; a redundância A/B depende do roteamento saudável do conjunto. O simulador temporal e a carga ao vivo usam motores distintos. Um sistema real precisa de benchmark, teste concorrente, teste de falhas e observação.

16. O repositório implementa uma plataforma de compras completa?

Resposta breve: publica o exemplo arquitetural, manifests e endpoints demonstrativos.

Aprofundamento: o catálogo é sintético, checkout retorna 501 e relay/worker não processam eventos. O schema explicita invariantes; o comportamento de domínio está descrito nas notas técnicas. O objetivo ao importar do GitHub é revisar evidências de arquitetura.

F. Visão de apresentação

17. Onde estão os outros componentes do mapa completo?

Resposta breve: a visão v10 agrupa as réplicas e mostra três jornadas; o mapa detalhado permanece na v9.

Aprofundamento: DNS, identidade, observabilidade, CI/CD e recuperação não foram descartados da arquitetura. Estão na revisão completa e nas notas técnicas. A simplificação serve para apresentar decisões em poucos minutos, sem misturar todas as relações de dados, controle e operação.